Ensemble Learning

Theoretical

Que - 1 Can we use Bagging for regression problems

Yes, **Bagging can be used for regression problems**. In regression, Bagging (Bootstrap Aggregating) trains multiple models on different random samples of the training data and combines their outputs by averaging instead of voting. This helps reduce variance and improves prediction stability. A common example is **Bagging Regressor**, where base models like decision trees are trained on different subsets and their predictions are averaged to get the final output.

Que - 2 What is the difference between multiple model training and single model training

Single model training means training **one algorithm on the entire dataset**, so the prediction is made by a single learned model, which is simple and fast but may suffer from overfitting or high bias/variance depending on the model. Multiple model training means training **more than one model (like in bagging, boosting, or ensembles)** on the same or different subsets of data and combining their outputs (by voting or averaging), which usually improves accuracy and stability but increases computation and complexity.

Que - 3 Explain the concept of feature randomness in Random Forest

Feature randomness in Random Forest means that at each split in a decision tree, the algorithm does not consider all available features; instead, it selects a **random subset of features** and chooses the best split only from that subset. This reduces correlation between trees, making each tree different and improving the overall performance of the ensemble. Because of this randomness, Random Forest becomes less prone to overfitting and gives better generalization compared to a single decision tree.

Que - 4 What is OOB (Out-of-Bag) Score

Out-of-Bag (OOB) score is a built-in validation method used in ensemble techniques like Random Forest. During training, each tree is built using a bootstrap sample of the data, meaning some samples are left out (called out-of-bag samples). These unused samples are then used to test that tree, and the combined performance across all trees gives the OOB score. It provides an unbiased estimate of model accuracy without needing a separate validation set.

Que - 5 How can you measure the importance of features in a Random Forest model

Feature importance in a Random Forest model can be measured by calculating how much each feature contributes to reducing impurity (like Gini impurity or entropy) across all the trees in the forest. Features that are used more often for splitting and lead to larger reductions in impurity are considered more important. In Python, this can be accessed using

the `feature_importances_` attribute of the trained Random Forest model, which gives a score for each feature indicating its relative importance in prediction.

Que - 6 Explain the working principle of a Bagging Classifier

A Bagging Classifier works on the principle of **Bootstrap Aggregation**, where multiple models are trained on different random samples of the training data created using sampling with replacement. Each model (usually decision trees) is trained independently, and because each model sees a slightly different dataset, they learn different patterns. For classification, the final output is obtained by **majority voting** from all models. This process reduces variance, improves stability, and helps prevent overfitting, especially for high-variance models like decision trees.

Que - 7 How do you evaluate a Bagging Classifier's performance

A Bagging Classifier's performance is evaluated using standard classification metrics on a test set or through cross-validation. Common evaluation measures include **accuracy**, **precision**, **recall**, and **F1-score**, which help assess how well the ensemble predicts different classes. Additionally, a **confusion matrix** can be used to analyze correct and incorrect predictions in detail. In some cases, **Out-of-Bag (OOB) score** is also used as an internal validation method, which estimates performance without needing a separate validation set.

Que -8 How does a Bagging Regressor work

A Bagging Regressor works by training multiple regression models on different bootstrap samples (random samples with replacement) of the training dataset. Each model is trained independently, usually using a base estimator like a decision tree. For prediction, the outputs of all models are combined by taking the **average of their predictions** instead of voting. This reduces variance, improves stability, and helps produce more accurate and robust regression results compared to a single model.

Que - 9 What is the main advantage of ensemble techniques

The main advantage of ensemble techniques is that they **improve model performance and generalization** by combining multiple models instead of relying on a single one. By aggregating predictions (through voting or averaging), ensemble methods reduce errors such as **overfitting, bias, and variance**, leading to more stable and accurate results.

Que - 10 What is the main challenge of ensemble methods

The main challenge of ensemble methods is their **increased complexity and computational cost**. Since multiple models are trained and combined, they require more time, memory, and processing power compared to a single model. They can also be harder to interpret, making it difficult to understand how final predictions are made.

Que - 11 Explain the key idea behind ensemble techniques

The key idea behind ensemble techniques is to **combine multiple individual models to produce a stronger and more accurate final prediction than any single model alone**. Instead of relying on one learner, ensemble methods aggregate the outputs of several

models (through voting in classification or averaging in regression). This improves performance by reducing errors such as variance, bias, and overfitting, leading to better generalization on unseen data.

Que - 12 What is a Random Forest Classifier

A Random Forest Classifier is an ensemble machine learning algorithm that builds **multiple decision trees** during training and combines their outputs to make the final prediction. Each tree is trained on a random subset of the data (bootstrap sampling) and uses a random subset of features at each split, which increases diversity among trees. For classification, the final output is decided by **majority voting** from all trees. This approach improves accuracy, reduces overfitting, and provides more stable and reliable predictions compared to a single decision tree.

Que 13 What are the main types of ensemble techniques

The main types of ensemble techniques are **Bagging, Boosting, and Stacking**. Bagging (Bootstrap Aggregating) trains multiple models independently on random subsets of data and combines their outputs to reduce variance. Boosting trains models sequentially, where each new model corrects the errors of the previous one, helping to reduce bias and improve accuracy. Stacking combines different types of models and uses another model (meta-learner) to learn how to best combine their predictions, improving overall performance.

Que 14 What is ensemble learning in machine learning

Ensemble learning in machine learning is a technique where **multiple models are combined to solve a problem and improve overall performance** instead of using a single model. Each model contributes its prediction, and the final output is obtained by combining them using methods like voting (classification) or averaging (regression). This approach helps reduce errors such as bias, variance, and overfitting, leading to more accurate and stable predictions compared to individual models.

Que 15 When should we avoid using ensemble methods

Ensemble methods should be avoided when the dataset is very small, as training multiple models can easily lead to overfitting and unreliable results. They are also not ideal when computational resources are limited, since ensemble techniques require more time and memory compared to a single model. Additionally, if interpretability is important, such as in medical or legal applications, simpler models are preferred because ensemble methods are harder to explain and understand.

Que 16 How does Bagging help in reducing overfitting

Bagging helps reduce overfitting by training multiple models on different **bootstrap samples** of the training data and then combining their predictions. Since each model sees a slightly different dataset, their errors are less correlated. When the outputs are averaged (regression) or voted (classification), the overall variance is reduced, making the final model more stable and less sensitive to noise or fluctuations in the training data.

Que 17 Why is Random Forest better than a single Decision Tree

Random Forest is better than a single Decision Tree because it combines the predictions of many trees trained on different random samples of the data and features. This reduces overfitting, which is common in a single decision tree that can easily memorize training data. By averaging or voting across multiple diverse trees, Random Forest improves accuracy, stability, and generalization on unseen data.

Que 18 What is the role of bootstrap sampling in Bagging

Bootstrap sampling in Bagging is used to create multiple different training datasets by randomly selecting samples from the original dataset **with replacement**. Each model in the ensemble is trained on a different bootstrap sample, which means some data points may appear multiple times while others may not appear at all. This introduces diversity among models, reduces correlation between them, and helps improve overall performance when their predictions are combined.

Que 19 What are some real-world applications of ensemble techniques

Ensemble techniques are widely used in real-world applications where high accuracy and reliability are important. They are used in **fraud detection** in banking to identify suspicious transactions, in **medical diagnosis** for predicting diseases like cancer more accurately, and in **recommendation systems** used by platforms like Netflix and Amazon. They are also used in **credit scoring**, **stock market prediction**, and **spam email detection**, where combining multiple models helps improve prediction stability and reduce errors.

Que 20 What is the difference between Bagging and Boosting

Bagging and Boosting are both ensemble techniques, but they work differently. **Bagging (Bootstrap Aggregating)** trains multiple models independently on random subsets of the data and combines their outputs using voting or averaging, mainly to reduce variance and prevent overfitting. **Boosting**, on the other hand, trains models sequentially, where each new model focuses on correcting the mistakes of the previous one, mainly to reduce bias and improve accuracy. Bagging gives equal importance to all models, while Boosting assigns more weight to harder-to-classify samples.

Practical

Que 21 Train a Bagging Classifier using Decision Trees on a sample dataset and print model accuracy

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
```

```
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Bagging Classifier with Decision Tree
model = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=10,
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Output -

```
Accuracy: 0.956140350877193
```

Que 22 Train a Bagging Regressor using Decision Trees and evaluate using Mean Squared Error (MSE)

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

# Create dataset
X, y = make_regression(n_samples=1000, n_features=5, noise=10, random_state=42)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Bagging Regressor with Decision Tree
model = BaggingRegressor(
    estimator=DecisionTreeRegressor(),
    n_estimators=10,
```

```
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("MSE:", mean_squared_error(y_test, y_pred))
```

Output -

```
MSE: 498.67
```

Que 23 Train a Random Forest Classifier on the Breast Cancer dataset and print feature importance scores

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Random Forest model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Feature importance
importances = model.feature_importances_

# Print feature importance scores
for name, score in zip(data.feature_names, importances):
    print(name, ":", score)
```

Output -

```
mean radius : 0.048703371737755234
mean texture : 0.013590877656998469
mean perimeter : 0.053269746128179675
mean area : 0.04755500886018552
mean smoothness : 0.007285327830663239
```

mean compactness : 0.013944325074050485
mean concavity : 0.06800084191430111
mean concave points : 0.10620998844591638
mean symmetry : 0.003770291819290666
mean fractal dimension : 0.0038857721093275
radius error : 0.02013891719419153
texture error : 0.004723988073894702
perimeter error : 0.01130301388178435
area error : 0.022406960160458473
smoothness error : 0.004270910110504497
compactness error : 0.005253215538990106
concavity error : 0.009385832251596627
concave points error : 0.003513255105598506
symmetry error : 0.004018418617722808
fractal dimension error : 0.00532145634222884
worst radius : 0.07798687515738047
worst texture : 0.021749011006763207
worst perimeter : 0.06711483267839194
worst area : 0.15389236463205394
worst smoothness : 0.010644205147280952
worst compactness : 0.020266035899623565
worst concavity : 0.031801595740040434
worst concave points : 0.14466326620735528
worst symmetry : 0.010120176131974357
worst fractal dimension : 0.005210118545497296

Que 24 Train a Random Forest Regressor and compare its performance with a single Decision Tree

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

# Create dataset
X, y = make_regression(n_samples=1000, n_features=10, noise=10, random_state=42)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

dt = DecisionTreeRegressor(random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)
dt_mse = mean_squared_error(y_test, dt_pred)

rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
```

```
rf_mse = mean_squared_error(y_test, rf_pred)

print("Decision Tree MSE:", dt_mse)
print("Random Forest MSE:", rf_mse)
```

Output -

```
Decision Tree MSE: 6432.419379494523
Random Forest MSE: 2727.0865628965244
```

Que 25 Compute the Out-of-Bag (OOB) Score for a Random Forest Classifier

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Random Forest with OOB enabled
model = RandomForestClassifier(
    n_estimators=100,
    oob_score=True,
    bootstrap=True,
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Print OOB Score
print("Out-of-Bag (OOB) Score:", model.oob_score_)
```

Output -

```
Out-of-Bag (OOB) Score: 0.9560439560439561
```

Que 26 Train a Bagging Classifier using SVM as a base estimator and print accuracy

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

```
# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Bagging with SVM as base estimator
model = BaggingClassifier(
    estimator=SVC(),
    n_estimators=10,
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Output -

```
Accuracy: 0.9473684210526315
```

Que 27 Train a Random Forest Classifier with different numbers of trees and compare accuracy

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Different numbers of trees
n_trees_list = [10, 50, 100, 200]
```

```
for n in n_trees_list:
    model = RandomForestClassifier(n_estimators=n, random_state=42)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    print("n_estimators =", n, "Accuracy =", acc)
```

Output -

```
n_estimators = 10 Accuracy = 0.956140350877193
n_estimators = 50 Accuracy = 0.9649122807017544
n_estimators = 100 Accuracy = 0.9649122807017544
n_estimators = 200 Accuracy = 0.9649122807017544
```

Que 28 Train a Bagging Classifier using Logistic Regression as a base estimator and print AUC score

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Bagging with Logistic Regression
model = BaggingClassifier(
    estimator=LogisticRegression(max_iter=1000),
    n_estimators=10,
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Predict probabilities (needed for AUC)
y_prob = model.predict_proba(X_test)[:, 1]

# AUC score
auc = roc_auc_score(y_test, y_prob)
```

```
print("ROC-AUC Score:", auc)
```

Output -

```
ROC-AUC Score: 0.9980347199475925
```

Que 29 Train a Random Forest Regressor and analyze feature importance scores

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor

# Create dataset
X, y = make_regression(
    n_samples=1000,
    n_features=5,
    noise=10,
    random_state=42
)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Random Forest Regressor
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Feature importance
importances = model.feature_importances_

# Print feature importance scores
for i, score in enumerate(importances):
    print("Feature", i, ":", score)
```

Output -

```
Feature 0 : 0.1706080362473906
Feature 1 : 0.5467231612362046
Feature 2 : 0.06419545735301468
Feature 3 : 0.14050526813482286
Feature 4 : 0.07796807702856733
```

Que 30 Train an ensemble model using both Bagging and Random Forest and compare accuracy

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Bagging Classifier
bag_model = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    random_state=42
)

bag_model.fit(X_train, y_train)
bag_pred = bag_model.predict(X_test)
bag_acc = accuracy_score(y_test, bag_pred)

# Random Forest Classifier
rf_model = RandomForestClassifier(
    n_estimators=50,
    random_state=42
)

rf_model.fit(X_train, y_train)
rf_pred = rf_model.predict(X_test)
rf_acc = accuracy_score(y_test, rf_pred)

# Comparison
print("Bagging Accuracy:", bag_acc)
print("Random Forest Accuracy:", rf_acc)
```

Output -

```
Bagging Accuracy: 0.956140350877193
Random Forest Accuracy: 0.9649122807017544
```

Que 31 Train a Random Forest Classifier and tune hyperparameters using GridSearchCV

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load dataset
```

```
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
rf = RandomForestClassifier(random_state=42)

# Hyperparameter grid
param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [None, 5, 10],
    "min_samples_split": [2, 5]
}

# GridSearchCV
grid = GridSearchCV(
    estimator=rf,
    param_grid=param_grid,
    cv=5,
    scoring="accuracy"
)

# Train
grid.fit(X_train, y_train)

# Best model
best_model = grid.best_estimator_

# Predict
y_pred = best_model.predict(X_test)

# Accuracy
print("Best Parameters:", grid.best_params_)
print("Test Accuracy:", accuracy_score(y_test, y_pred))
```

Output -

```
Best Parameters: {'max_depth': None, 'min_samples_split': 2, 'n_estimators': 200}
Test Accuracy: 0.9649122807017544
```

Que 32 Train a Bagging Regressor with different numbers of base estimators and compare performance

```
from sklearn.datasets import make_regression
```

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

# Create dataset
X, y = make_regression(
    n_samples=1000,
    n_features=10,
    noise=10,
    random_state=42
)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Different numbers of estimators
n_estimators_list = [5, 10, 50, 100]

for n in n_estimators_list:
    model = BaggingRegressor(
        estimator=DecisionTreeRegressor(),
        n_estimators=n,
        random_state=42
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    print("n_estimators =", n, "MSE =", mse)
```

Output -

```
n_estimators = 5 MSE = 4281.909633962362
n_estimators = 10 MSE = 3385.579200047012
n_estimators = 50 MSE = 2783.278363752002
n_estimators = 100 MSE = 2721.0727695853593
```

Que 33 Train a Random Forest Classifier and analyze misclassified samples

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Load dataset
data = load_breast_cancer()
X = data.data
```

```
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Random Forest
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Analyze misclassified samples
print("Misclassified Samples:")

for i in range(len(y_test)):
    if y_test[i] != y_pred[i]:
        print("Index:", i,
              "Actual:", y_test[i],
              "Predicted:", y_pred[i])
```

Output -

```
Misclassified Samples:
Index: 8 Actual: 1 Predicted: 0
Index: 20 Actual: 0 Predicted: 1
Index: 77 Actual: 0 Predicted: 1
Index: 82 Actual: 0 Predicted: 1
```

Que 34Train a Bagging Classifier and compare its performance with a single Decision Tree Classifier

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Single Decision Tree
```

```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)
dt_acc = accuracy_score(y_test, dt_pred)

# Bagging Classifier
bag = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    random_state=42
)

bag.fit(X_train, y_train)
bag_pred = bag.predict(X_test)
bag_acc = accuracy_score(y_test, bag_pred)

# Comparison
print("Decision Tree Accuracy:", dt_acc)
print("Bagging Classifier Accuracy:", bag_acc)
```

Output -

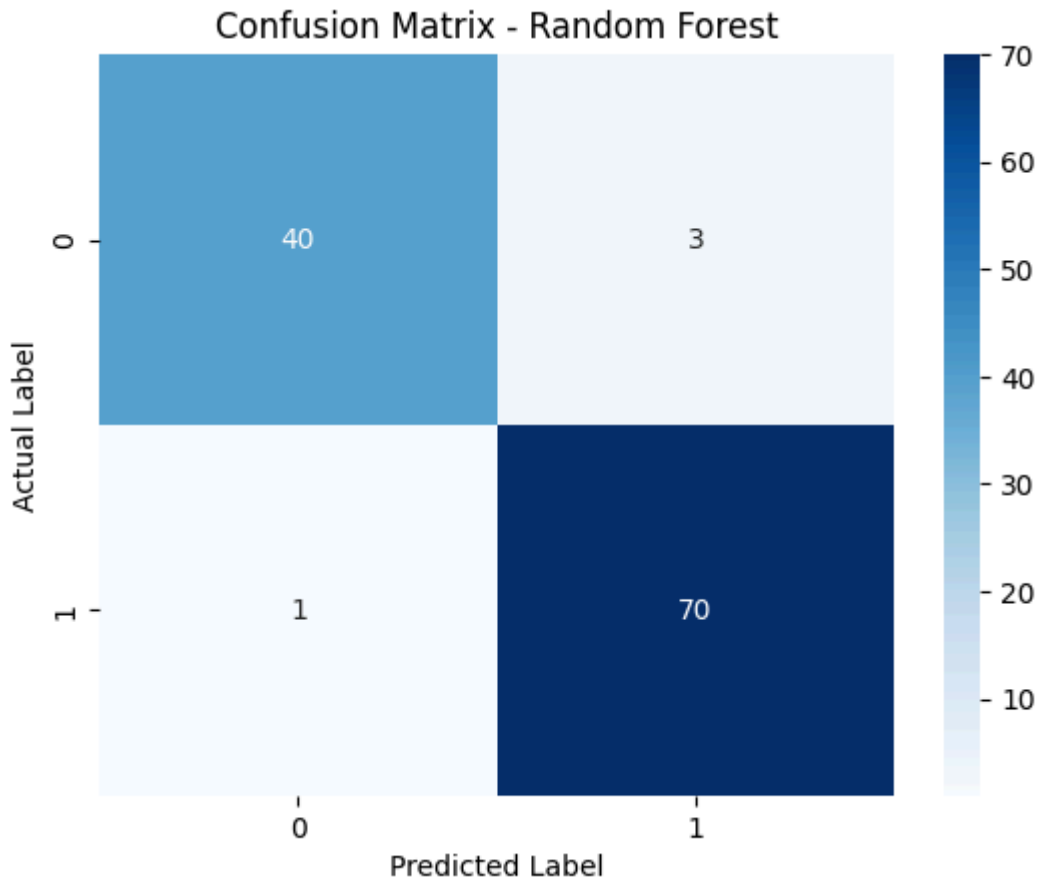
```
Decision Tree Accuracy: 0.9473684210526315
Bagging Classifier Accuracy: 0.956140350877193
```

Que 35 Train a Random Forest Classifier and visualize the confusion matrix

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Random Forest
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)
```

```
# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Visualization
plt.figure()
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix - Random Forest")
plt.show()
```

Output -



Que 36 Train a Stacking Classifier using Decision Trees, SVM, and Logistic Regression, and compare accuracy

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import StackingClassifier
```

```
from sklearn.metrics import accuracy_score

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Base models
base_models = [
    ('dt', DecisionTreeClassifier()),
    ('svm', SVC(probability=True)),
]

# Stacking Classifier
model = StackingClassifier(
    estimators=base_models,
    final_estimator=LogisticRegression(max_iter=1000)
)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
print("Stacking Classifier Accuracy:", accuracy_score(y_test, y_pred))
```

Output -

```
Stacking Classifier Accuracy: 0.956140350877193
```

Que 37 Train a Random Forest Classifier and print the top 5 most important features

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
import numpy as np

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target
feature_names = data.feature_names
```

```
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Random Forest
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Feature importance
importances = model.feature_importances_

# Get top 5 features
indices = np.argsort(importances)[::-1][:5]

print("Top 5 Important Features:")
for i in indices:
    print(feature_names[i], ":", importances[i])
```

Output -

```
Top 5 Important Features:
worst area : 0.15389236463205394
worst concave points : 0.14466326620735528
mean concave points : 0.10620998844591638
worst radius : 0.07798687515738047
mean concavity : 0.06800084191430111
```

Que 38 Train a Bagging Classifier and evaluate performance using Precision, Recall, and F1-score

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Bagging Classifier
model = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
```

```
    n_estimators=50,
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluation
print(classification_report(y_test, y_pred))
```

Output -

```
              precision    recall  f1-score   support

           0       0.95      0.93      0.94        43
           1       0.96      0.97      0.97        71

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```

Que 39 Train a Random Forest Classifier and analyze the effect of max_depth on accuracy

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Different max_depth values
depth_values = [2, 5, 10, 20, None]

for depth in depth_values:
    model = RandomForestClassifier(
        n_estimators=100,
        max_depth=depth,
        random_state=42
```

```
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    print("max_depth =", depth, "Accuracy =", acc)
```

Output -

```
max_depth = 2 Accuracy = 0.9649122807017544
max_depth = 5 Accuracy = 0.9649122807017544
max_depth = 10 Accuracy = 0.9649122807017544
max_depth = 20 Accuracy = 0.9649122807017544
max_depth = None Accuracy = 0.9649122807017544
```

Que 40 Train a Bagging Regressor using different base estimators (DecisionTree and KNeighbors) and compare performance

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error

# Create dataset
X, y = make_regression(
    n_samples=1000,
    n_features=10,
    noise=10,
    random_state=42
)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Bagging with Decision Tree
bag_dt = BaggingRegressor(
    estimator=DecisionTreeRegressor(),
    n_estimators=50,
    random_state=42
)

bag_dt.fit(X_train, y_train)
pred_dt = bag_dt.predict(X_test)
mse_dt = mean_squared_error(y_test, pred_dt)
```

```
# Bagging with KNN
bag_knn = BaggingRegressor(
    estimator=KNeighborsRegressor(),
    n_estimators=50
)

bag_knn.fit(X_train, y_train)
pred_knn = bag_knn.predict(X_test)
mse_knn = mean_squared_error(y_test, pred_knn)

# Comparison
print("Bagging with Decision Tree MSE:", mse_dt)
print("Bagging with KNN MSE:", mse_knn)
```

Output -

```
Bagging with Decision Tree MSE: 2783.278363752002
Bagging with KNN MSE: 3545.316243847508
```

Que 41 Train a Random Forest Classifier and evaluate its performance using ROC-AUC Score

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Random Forest Classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predict probabilities (needed for ROC-AUC)
y_prob = model.predict_proba(X_test)[:, 1]

# ROC-AUC score
auc = roc_auc_score(y_test, y_prob)

print("ROC-AUC Score:", auc)
```

Output -

```
ROC-AUC Score: 0.9952505732066819
```

Que 42 Train a Bagging Classifier and evaluate its performance using cross-validation

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Bagging Classifier
model = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    random_state=42
)

# Cross-validation (5-fold)
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')

# Output results
print("Cross-validation scores:", scores)
print("Average Accuracy:", scores.mean())
```

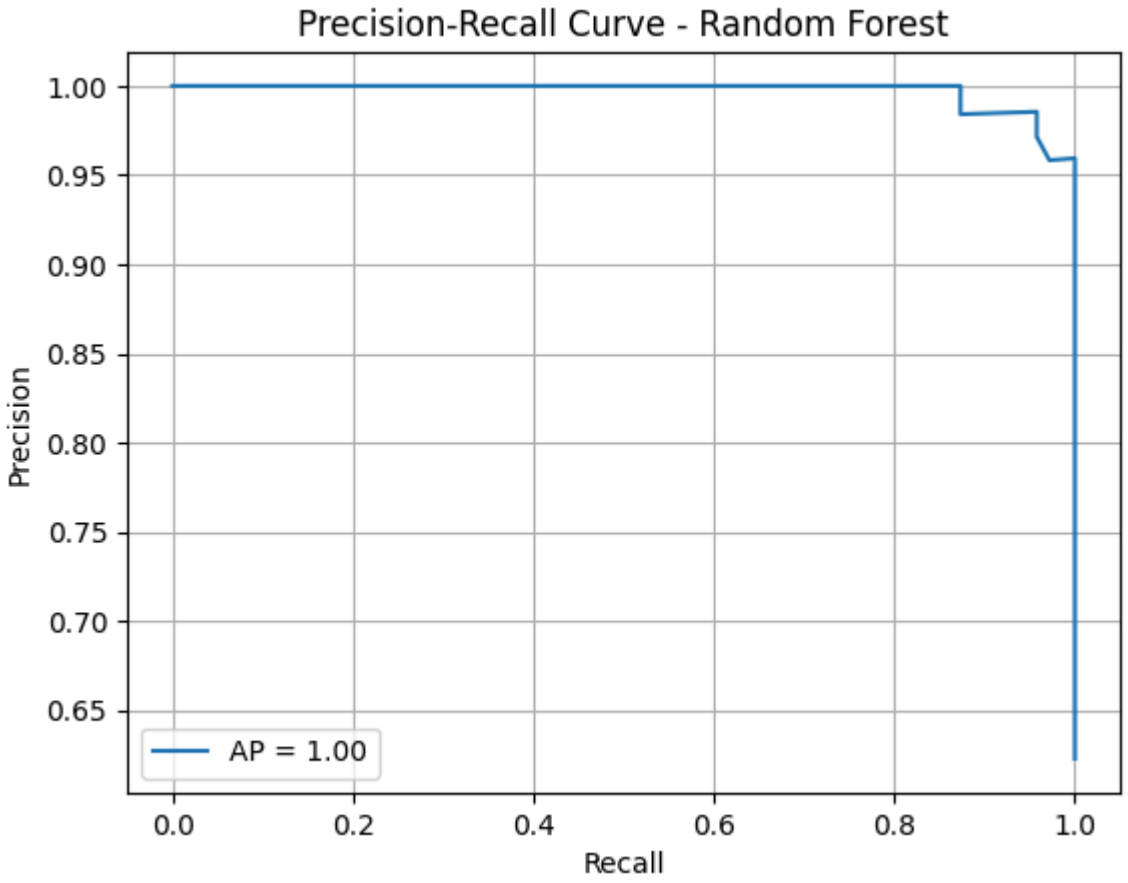
Output -

```
Cross-validation scores: [0.9122807  0.92105263 0.98245614 0.95614035 1.
]
Average Accuracy: 0.9543859649122808
```

Que 43 Train a Random Forest Classifier and plot the Precision-Recall curve

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import precision_recall_curve, average_precision_score
import matplotlib.pyplot as plt

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Train Random Forest Classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Get probability scores
y_scores = model.predict_proba(X_test)[:, 1]

# Precision-Recall values
precision, recall, _ = precision_recall_curve(y_test, y_scores)

# Average Precision Score
ap = average_precision_score(y_test, y_scores)

# Plot curve
plt.figure()
plt.plot(recall, precision, label=f"AP = {ap:.2f}")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve - Random Forest")
plt.legend()
plt.grid()
plt.show()
```



Que 44 Train a Stacking Classifier with Random Forest and Logistic Regression and compare accuracy

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load dataset
data = load_breast_cancer()
X = data.data
y = data.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Base models
estimators = [
    ('rf', RandomForestClassifier(n_estimators=100, random_state=42))
]

# Stacking Classifier
model = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression(max_iter=1000)
)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
print("Stacking Classifier Accuracy:", accuracy_score(y_test, y_pred))
```

Output -

```
Stacking Classifier Accuracy: 0.9649122807017544
```

Que 45 Train a Bagging Regressor with different levels of bootstrap samples and compare performance

```
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error
```

```
# Create dataset
X, y = make_regression(
    n_samples=1000,
    n_features=10,
    noise=10,
    random_state=42
)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Different bootstrap sample sizes
sample_sizes = [0.5, 0.7, 0.8, 1.0]

for size in sample_sizes:
    model = BaggingRegressor(
        estimator=DecisionTreeRegressor(),
        n_estimators=50,
        max_samples=size,
        bootstrap=True,
        random_state=42
    )

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    print("Bootstrap sample size =", size, "MSE =", mse)
```

Output-

```
Bootstrap sample size = 0.5 MSE = 2755.9547253361497
Bootstrap sample size = 0.7 MSE = 2968.0121364038973
Bootstrap sample size = 0.8 MSE = 2823.2943559433006
Bootstrap sample size = 1.0 MSE = 2783.278363752002
```